PYTHON

Learning Python #001: Functions, Variables, and My First CLI Tool

Starting my journey into software and AI engineering with Harvard CS50P, Python fundamentals, and my first command-line application.

David Becker · October 20, 2026 · 5 min read

Simpyll · https://simpyll.com/blog/001-python-fundamentals

Learning Python #001: Functions, Variables, and My First CLI Tool

I'm starting something new: a long-term journey to become a proficient software engineer, with the ultimate goal of specializing in artificial intelligence and machine learning.

I've experimented with programming before. I've used Arch Linux, spent time working in the terminal, written shell scripts, and played around with Python. But there's a significant difference between understanding what code generally does and being able to design, write, debug, and maintain software independently.

That's the gap I'm working to close.

Rather than jumping straight into AI frameworks or relying on AI to generate everything for me, I've decided to start with the fundamentals and build something at every step.

My first stop is Harvard's CS50 Introduction to Programming with Python (https://cs50.harvard.edu/python/).

Lesson 001: Functions and Variables

The first lecture covers some of the most fundamental concepts in programming: functions, variables, user input, strings, and how programs execute instructions.

These concepts are relatively simple individually, but they're the building blocks for everything that comes afterward.

1. Printing Output

The first function I worked with was `print()`.

```python
print("Hello, world!")
```

It displays text in the terminal.

Simple enough, but it introduces an important concept: functions allow us to execute predefined behavior without having to implement that behavior ourselves.

2. Variables and User Input

Next came variables and the `input()` function.

```python
name = input("What's your name? ")
print(f"Hello, {name}!")
```

The `input()` function waits for the user to enter something and returns that value as a string.

That value is then assigned to the variable `name`.

The second line uses an f-string, which allows variables and expressions to be embedded directly inside a string.

Instead of hardcoding the same output every time, the program can now respond dynamically to user input.

3. Working With Strings

One particularly useful method is `.strip()`, which removes leading and trailing whitespace.

```python
university = input("University: ").strip()
```

If someone enters:

```text
   Graceland University
```

The stored value becomes:

```
Graceland University
```

I also experimented with methods like `.title()`, `.lower()`, and `.upper()`.

These small operations become important when working with user-generated input, search queries, and eventually larger datasets.

What I Built: NP Program Collector

To reinforce the concepts, I built a small command-line application called **NP Program Collector**.

The idea came from another project I'm working on: RNtoNP.com (https://rntonp.com), a platform intended to help nurses explore educational pathways and compare nurse practitioner programs.

Eventually, I'd like to build systems that automatically collect, organize, and analyze information about universities and their programs.

But for now, I'm starting with something much simpler.

The application asks the user to enter a university, a nurse practitioner program, and an annual tuition amount. It then displays the information in a formatted summary.

The Code

Here's the complete Python program:

```python
def main():
    print("NP Program Collector")
    print("--------------------")

    university = input("University: ").strip()
    program = input("NP Program: ").strip()
    tuition = input("Annual Tuition: ").strip()

    print()
    print("--- Program Summary ---")
    print(f"University: {university}")
    print(f"Program: {program}")
    print(f"Annual Tuition: ${tuition}")
```

```
if __name__ == "__main__":
    main()
```

Running the Program

I saved the application as `program_collector.py` and ran it from the terminal:

bash

```
python program_collector.py
```

Here's an example of the program in action:

text

```
NP Program Collector
--------------------
University: Graceland University
NP Program: AGACNP
Annual Tuition: 15000

--- Program Summary ---
University: Graceland University
Program: AGACNP
Annual Tuition: $15000
```

It's a small application, but it represents a complete cycle: accepting input, processing values, and producing output.

Why Use a main() Function?

I organized the program inside a function called `main()`.

python

```
def main():
    print("NP Program Collector")
```

The `def` keyword defines a function.

Functions allow code to be organized into reusable, logically related blocks.

At the bottom of the program, I added:

```python
if __name__ == "__main__":
    main()
```

This tells Python to execute `main()` when the file is run directly, while avoiding automatically running it if the file is imported as a module.

It's a useful convention as programs become more complex.

What I Learned Beyond the Syntax

The biggest takeaway wasn't any individual Python function. It was starting to think about programs as systems that transform inputs into outputs.

My application follows a straightforward sequence:

1. Display an introduction.
2. Collect information from the user.
3. Clean the input.
4. Store the values.
5. Present the results.

That basic structure appears everywhere in software engineering.

An API accepts a request, processes information, and returns a response. A machine learning model receives input data, performs computations, and produces predictions.

The implementations become dramatically more sophisticated, but the underlying idea of transforming information remains.

What Could Be Improved?

The application works for its intended demonstration, but it's far from production-ready.

For example, the tuition field accepts any text.

Someone could enter:

```text
Annual Tuition: banana
```

And the program would happily display:

text

```
Annual Tuition: $banana
```

There's no input validation.

The application also doesn't save information between executions. Once the program closes, the data is gone.

And although it accepts information about a university, it doesn't retrieve or verify anything automatically.

These limitations create opportunities for future lessons.

As I learn more Python, I plan to expand the application with input validation, error handling, persistent storage, automated data collection, and eventually a database.

The goal is to let the project evolve alongside my skills.

The Bigger Picture: From Python to AI Engineering

This is the first step in a much larger learning roadmap.

My long-term curriculum will cover computer science fundamentals, algorithms, mathematics, backend development, distributed systems, machine learning, deep learning, and AI infrastructure.

Eventually, I want to understand how modern AI systems actually work, from the mathematics behind neural networks to the infrastructure required to train and serve large language models.

That means learning to implement things myself rather than relying exclusively on existing frameworks or AI-generated code.

I'm documenting the process publicly for two reasons.

First, explaining what I've learned forces me to organize my understanding.

Second, I want to build a portfolio that demonstrates real technical progression, including the mistakes, debugging, and improvements along the way.

What's Next?

The next lesson focuses on conditionals and Boolean expressions.

I'll learn how programs make decisions using `if`, `elif`, `else`, and comparison operators.

I'll also expand NP Program Collector so it can recognize invalid inputs and respond appropriately.

For now, the first milestone is complete: a small, working Python application built around the fundamentals.

Lesson 001 complete. On to the next one.

Resources

- Harvard CS50P — Introduction to Programming with Python (https://cs50.harvard.edu/python/)
- CS50P Week 0 — Functions and Variables (https://cs50.harvard.edu/python/2022/weeks/0/)
- Python 3 Documentation (https://docs.python.org/3/)

Part 1 of my Zero to AI Engineer series — documenting the journey from programming fundamentals to advanced AI engineering, one project at a time.